

SYNOPSIS OF EXPLORATORY PROJECT (EP)

ON

AI Loan Management System

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Dhruv Sharma, 2310990410

Dhrupad Kapoor, 2310990409

Priyansh Thakur, 2310991349

Supervised By:

Dr. Rajat Takkar

Assistant Professor

Department of Computer Science & Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page Number
	Declaration	3
	Abstract	4
1.	Introduction 1.1 Background and Context 1.2 Problem Statement 1.3 Objectives of the Project 1.4 Significance of the Project 1.5 Scope and Limitations	5-7
2.	Methodology 2.1 Technical Approach 2.2 System Architecture 2.3 Functional Modules Overview 2.4 Workflow Diagram (Optional)	8-9
3.	Tools And Technologies 3.1 Programming Languages 3.2 Frontend Technologies 3.3 Backend Technologies 3.4 Database 3.5 Authentication and Security 3.6 APIs and Services 3.7 Development and Deployment 3.8 IDEs and Development Platforms 3.9 Technology Stack Summary	10-12
4.	Project Plan 4.1 Module-wise Feature Breakdown 4.2 Phase-wise Development Plan (Phases 1-10) 4.3 Project Development Flow 4.4 Week-wise Development Timeline 4.5 Expected Outcomes 4.6 Risk Factors and Assumptions	13-17
5.	References / Literature Review Table	18-19

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**AI Loan Management System**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Rajat Takkar. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Chitkara University, Rajpura

Date: 24-08-2026

Dhruv Sharma, 231090410

Dhrupad Kapoor, 2310990409

Priyansh Thakur, 2310991349

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Rajat Takkar

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

Abstract

I developed an AI-enhanced Loan Management System to streamline loan application, tracking, and user assistance. This full-stack web application integrates modern technologies: a React frontend (with Vite and Tailwind CSS) connects via REST APIs to a Node.js/Express backend. We used PostgreSQL with Prisma ORM for secure data management, Google's Gemini API for generative AI features, and Nodemailer (SMTP via Gmail) for email verification and notifications. The system supports user registration and JWT-based authentication, loan creation and scheduling, repayment tracking, and AI-powered financial advice. In this report I describe the project's purpose, architecture, methodology, implementation highlights, outcomes, security considerations, limitations, and future work.

In our implementation, the React/Vite frontend provides a responsive UI for borrowers and administrators. The backend (Node/Express) exposes endpoints for user accounts, loans, and finances. We leveraged PostgreSQL as the database – “a powerful, open source object-relational database system... with a strong reputation for reliability” – and managed schema with Prisma ORM, which offers automated migrations and type-safe querying. This modular architecture allows the frontend to send JSON requests to the backend, where Prisma generates tables and enforces data integrity. Google Gemini's generative AI (via the Gemini API) was integrated to assist users with financial insights; as Google describes it, “Gemini API is the fastest path from prompt to production... integrate these generative models into your applications to generate text and images”. For email features (password reset, notifications), we used Nodemailer, “the most popular email sending library for Node.js” with Gmail's SMTP (requiring a Google App Password).

Our methodology was iterative and modular: I first set up the database and Prisma schema, then built the backend endpoints for user and loan management, and finally developed the React interface. Key implementation steps included configuring JWT tokens for secure login, using Prisma to automatically create and migrate database tables, and wiring up Google Gemini's REST API for AI queries. Nodemailer was configured with secure SMTP settings in the `.env` file, as recommended for production use. These components were tested using Postman and manual flows.

1. Introduction

1.1. Background and Context

The increasing use of digital financial services has created a need for efficient systems that can manage loan-related activities in a structured and accessible manner. Traditional loan management processes may involve multiple manual steps for maintaining user information, loan records, financial details, and communication. Such processes can become time-consuming and difficult to manage when the volume of information increases.

The **AI Loan Management System** is developed as a full-stack web application to provide a centralized platform for managing loan and financial information. The system combines a web-based user interface with backend APIs, a relational database, authentication mechanisms, AI-based assistance, and email services. The project follows the Exploratory Project requirements specified in the provided report format, which emphasizes documenting the methodology, technologies, implementation, outcomes, and future scope of the developed system.

The application uses **React.js, Vite, and Tailwind CSS** for the frontend and **Node.js with Express.js** for backend development. **PostgreSQL** is used for persistent data storage, with **Prisma ORM** providing database modelling and management. The system also integrates **Google Gemini AI** for AI-based financial assistance and **Nodemailer with Gmail SMTP** for email-related functionality. Authentication is implemented using JWT to provide protected access to application resources.

1.2. Problem Statement

Managing loan information requires the systematic handling of users, loan applications, financial records, and related communication. When these activities are handled through disconnected or manual processes, it can become difficult to maintain consistent records, track loan-related information, and provide timely assistance to users.

Another challenge is providing users with an accessible platform where they can interact with loan and financial information while maintaining appropriate authentication and data management. Conventional systems may also lack integrated AI-based assistance that can help users obtain relevant information during their interaction with the application.

Therefore, the problem addressed by this project is the development of a **centralized and secure web-based loan management system** that can organize loan-related information, provide authenticated access, support financial management, and integrate AI-based assistance within a single application.

1.3. Objectives of the Project

- To develop a **web-based platform** for managing loan-related information.
- To provide a structured mechanism for **user registration and authentication**.
- To manage and store **loan and financial information** using a relational database.
- To develop backend APIs for communication between the frontend and database.
- To integrate **PostgreSQL and Prisma ORM** for reliable data management.
- To integrate **Google Gemini AI** for providing AI-based financial assistance.
- To implement **email functionality** using Nodemailer and Gmail SMTP.
- To provide a responsive and user-friendly interface using React and Tailwind CSS.
- To maintain a modular architecture that allows different system components to work together efficiently.

1.4. Significance of the Project

The project is significant because it demonstrates how modern web technologies and AI services can be combined to develop a practical financial management application. Instead of treating authentication, database management, loan information, AI assistance, and email communication as separate systems, the project brings these components together within a single application.

The use of **JWT authentication** provides a mechanism for protecting authenticated resources, while PostgreSQL and Prisma provide structured management of application data. The integration of Gemini AI demonstrates the potential of generative AI in providing interactive financial assistance.

From a technical perspective, the project also demonstrates the implementation of a complete **frontend-backend-database architecture**. It provides practical experience in REST API development, database modelling, authentication, third-party API integration, and email service integration.

The project therefore serves both as a functional prototype and as an exploration of how contemporary software development technologies can be applied to the domain of loan management.

1.5. Scope and Limitations

The scope of the project includes the development of a full-stack web application capable of supporting:

- User registration and authentication.
- Secure access through JWT-based authentication.
- Loan-related information and management.
- Financial information management.
- PostgreSQL-based data storage.
- Prisma-based database operations and migrations.
- REST API communication between frontend and backend.
- AI-based assistance through Google Gemini.
- Email functionality through Nodemailer and Gmail SMTP.
- A responsive web interface developed using React and Tailwind CSS.

The current project is primarily a **working prototype** and has limitations that should be considered before deployment in a real-world financial environment.

- It does not currently represent a complete production-level banking or lending platform.
- Advanced credit-risk assessment and automated loan approval are outside the current implementation scope.
- Online payment-gateway integration is not part of the current system.
- AI functionality depends on the availability and configuration of the Gemini API.
- Email functionality depends on correctly configured SMTP credentials and connectivity.
- Additional security, scalability, performance, and extensive real-world testing would be required before production deployment.
- Integration with external banking or credit-bureau systems is not currently included.

2. Methodology

2.1. Technical Approach

The AI Loan Management System follows a **modular full-stack development approach**. The frontend is developed using React with Vite and Tailwind CSS, while the backend is implemented using Node.js and Express.js. Communication between the frontend and backend is handled through REST APIs. PostgreSQL is used for persistent data storage, with Prisma ORM managing database models, migrations, and queries. The development approach separates presentation, business logic, and data management, making the system easier to maintain and extend.

The methodology also incorporates **JWT-based authentication**, Gemini API integration for AI-assisted financial functionality, and Nodemailer with Gmail SMTP for email-related services. Configuration details such as database credentials, API keys, and SMTP credentials are maintained through environment variables rather than being directly embedded in the application.

2.2. System Architecture

The system follows a **three-layer architecture** consisting of the presentation layer, application/backend layer, and database layer. Users interact with the React-based frontend, which sends requests to the Express.js backend. The backend processes requests, performs authentication and business operations, communicates with PostgreSQL through Prisma, and interacts with external services such as Gemini and Gmail SMTP when required.

Architecture

Flow:

User → React/Vite Frontend → REST API → Node.js/Express Backend → Prisma ORM → PostgreSQL Database

External services such as **Google Gemini AI** and **Gmail SMTP/Nodemailer** are connected to the backend according to the functionality being performed.

2.3. Functional Modules Overview

- **User Authentication Module** – Registration, login, JWT-based authentication, and protected access.
- **User Management Module** – Management of user-related information and account data.
- **Loan Management Module** – Handling loan-related information and loan lifecycle operations.
- **Finance Module** – Management and tracking of financial information associated with users and loans.
- **AI Assistance Module** – Integration with Gemini API for AI-powered financial assistance.
- **Email Service Module** – Email communication through Nodemailer and Gmail SMTP.
- **Database Management Module** – Persistent storage and data access using PostgreSQL and Prisma ORM.

2.4. Workflow Diagram (Optional)

The overall workflow can be represented as:

User Registration/Login → Authentication → Dashboard → Loan/Finance Operations → Backend Processing → Database Storage → Response to User

For AI-related operations, the workflow extends to:

User Request → Frontend → Backend → Gemini API → AI Response → Frontend

3. Tools and Technologies

3.1. Programming Languages

The project primarily uses **JavaScript** for both frontend and backend development. React components and frontend logic are implemented using JavaScript, while the **backend uses JavaScript with Node.js and Express.js**. This provides a consistent programming environment across the application and simplifies communication between **frontend components and backend APIs**.

3.2. Frontend Technologies

- **React.js** – Component-based user interface development.
- **Vite** – Fast frontend development and build environment.
- **Tailwind CSS** – Responsive and utility-based styling.
- **React Components** – Reusable UI elements and structured application design.
- **REST API Integration** – Communication between frontend and backend.
- **Responsive UI** – User-friendly interface across different screen sizes.

3.3. Backend Technologies

- **Node.js** – Server-side JavaScript runtime.
- **Express.js** – Backend framework for REST API development.
- **Rest APIs** – Handling frontend–backend communication.
- **JavaScript** – Backend application logic and processing.
- **Middleware** – Request processing and backend functionality.
- **Environment Variables** – Configuration of sensitive backend settings.

3.4. Database

- **Prisma ORM** – Database interaction and query management.
- **PostgreSQL** – Primary relational database.
- **Prisma Schema** – Defines database models and relationships.
- **Database Migrations** – Maintains and updates database structure.
- **Persistent Storage** – Stores user, loan and financial information.
- **Prisma Studio** – Useful for viewing database records during development.

3.5. Authentication and Security

- **JWT Authentication** – Secure user authentication.
- **Access Tokens** – Used for authenticated API requests.
- **Protected Routes** – Restricts access to authorized users.
- **Environment Variables** – Protects API keys and database credentials.
- **Authentication Middleware** – Validates user authorization.
- **Secure Configuration** – Sensitive credentials are kept outside source code.

3.6. APIs and Services

- **Google Gemini API** – Provides AI-powered financial assistance.
- **Nodemailer** – Handles email communication from the backend.
- **Gmail SMTP** – Used as the email delivery service.
- **REST APIs** – Connect frontend operations with backend services.
- **External API Integration** – Extends the functionality of the application.
- **Environment-based API Configuration** – API credentials are managed through `.env`.

3.7. Development and Deployment

- **npm** – Package and dependency management.
- **Vite Development Server** – Used for frontend development.
- **Node.js Development Server** – Used to run the backend.
- **Prisma Migrations** – Used to initialize and update the database.
- **Environment Configuration** – Separate configuration through `.env` files.
- **Local Development Environment** – Frontend, backend and PostgreSQL run as integrated components.

3.8. IDEs and Development Platforms

- **Visual Studio Code** – Primary IDE for development.
- **pgAdmin** – Used for PostgreSQL database administration.
- **Prisma Studio** – Used for inspecting database records.
- **Postman** – Used for testing backend APIs.
- **Node.js/npm Environment** – Used to run and manage the project.

3.9. Technology Stack Summary

JavaScript, React.js, Vite, Tailwind CSS, Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, REST APIs, Weather API, Cloudinary, Git, GitHub, Render and Vercel.

4. Project Plan

The development of the **AI Loan Management System** is divided into systematic phases covering requirement analysis, system design, frontend and backend development, database integration, authentication, AI and email service integration, testing, and final deployment. The following plan describes the major modules and development stages of the project.

4.1. Module-wise Feature Breakdown

- **Authentication Module** – User registration, login, JWT-based authentication, and protected access.
- **User Management Module** – Management of user profiles and account-related information.
- **Loan Management Module** – Loan application, loan information, status tracking, and related operations.
- **Finance Module** – Management and tracking of financial information associated with users and loans.
- **AI Assistance Module** – Gemini API integration for AI-powered financial assistance and insights.

4.2. Phase 1 – Requirement Analysis

The first phase focuses on identifying the requirements of a digital loan management platform. The major requirements include secure user authentication, user and financial data management, loan-related operations, AI-assisted functionality, email communication, and reliable database storage. The requirements are organized to provide a centralized system for managing loan and financial activities.

4.3. Phase 2 – System Design

The system is designed using a **client-server architecture** in which the frontend, backend, database, and external services operate as connected components. The React frontend provides the user interface, while the Node.js and Express.js backend manages application logic and REST APIs. PostgreSQL is selected for persistent storage, with Prisma ORM managing database operations.

4.4. Phase 3 – Frontend Development

The frontend is developed using **React.js, Vite, and Tailwind CSS**. The interface is organized into reusable components and application pages required for user interaction. The frontend communicates with backend REST APIs to submit requests, retrieve information, and display application data. Responsive design principles are used to provide a consistent user experience across different screen sizes.

4.5. Phase 4 – Backend Development

The backend is developed using **Node.js and Express.js**. REST API endpoints are implemented to handle authentication, user operations, loan-related functionality, and financial data processing. Backend middleware is used for request processing and authentication, while the application logic manages communication between the frontend, database, and external services.

4.6. Phase 5 – Database Integration

PostgreSQL is integrated as the primary database, while **Prisma ORM** is used for database access and schema management. Prisma migrations are used to create and update the database structure. The database stores the information required for users, loans, financial records, and authentication-related operations in a structured manner.

4.7. Phase 6 – Authentication and Security

A JWT-based authentication mechanism is implemented to provide secure access to the application.

- User registration and login are implemented through backend APIs.
- JWT tokens are used to authenticate users.
- Protected routes restrict unauthorized access.
- Authentication middleware validates requests to protected resources.

- Sensitive credentials are stored using environment variables.
- Database access is managed through Prisma ORM.
- API and service credentials are kept outside the source code.

4.8. Phase 7 – External API Integration

The system integrates external services to extend its functionality. **Google Gemini API** is used for AI-powered financial assistance, allowing the application to provide intelligent responses related to financial and loan-related interactions. **Nodemailer** is integrated with Gmail SMTP to provide application email functionality. API keys and SMTP credentials are configured through environment variables.

4.9. Phase 8 – Testing and Debugging

The application is tested at both frontend and backend levels to verify the major system operations.

- User registration and login functionality.
- JWT authentication and protected routes.
- Loan-related API operations.
- Database connectivity and Prisma operations.
- Frontend and backend API communication.
- Gemini API integration.
- Email service and SMTP configuration.

4.10. Phase 9 – Deployment

After development and testing, the application can be prepared for deployment by configuring the frontend, backend, database, and external service credentials in the target environment. Environment variables are configured for the database connection, JWT secrets, Gemini API key, and SMTP credentials. The current project setup primarily supports local development and does not establish a specific production hosting platform in the provided project materials.

4.11. Phase 10 – Future Enhancement

The system can be further extended with additional financial and loan-management capabilities. Possible enhancements include AI-based loan eligibility analysis, credit-risk assessment, automated loan recommendations, online repayment facilities, advanced financial dashboards, mobile application support, and integration with external financial or banking services.

4.12. Project Development Flow

Requirement Analysis → System Design → Frontend Development → Backend & API Development → PostgreSQL & Prisma Integration → Authentication & Security → AI & Email Integration → Testing → Deployment → Future Enhancements

4.13. Week-wise Development Timeline

Module	Planned Weeks	Expected Deliverables
Module 1: Requirement Analysis	Week 1	Finalized requirements and feature list
Module 2: System Design	Week 2	System architecture and database design
Module 3: Frontend Development	Week 3–4	React-based application interface
Module 4: Backend Development	Week 4–5	REST APIs and backend functionality
Module 5: Database Integration	Week 5–6	PostgreSQL database with Prisma
Module 6: Authentication and Security	Week 6	JWT authentication and protected routes

Module 7: AI & Email Integration	Week 7	Gemini and Nodemailer integration
Module 8: Testing and Debugging	Week 8	Tested application functionality
Module 9: Deployment	Week 9	Configured application environment
Final Documentation and Report	Week 10	Completed synopsis/project report

4.14. Expected Outcomes

- A functional web-based platform for managing **users, loans, and financial information**.
- Secure user registration, login, and protected application access.
- Centralized management of loan-related and financial records.
- AI-powered assistance through **Google Gemini API**.
- Email communication through **Nodemailer and Gmail SMTP**.
- Structured and reliable data management using **PostgreSQL and Prisma ORM**.

4.15. Risk Factors and Assumptions

- Dependence on external services such as the **Gemini API and Gmail SMTP** may affect related functionality if the services are unavailable or incorrectly configured.
- The application requires a working PostgreSQL database and correctly configured environment variables.
- AI functionality depends on the availability and validity of the configured Gemini API key.
- Email functionality depends on valid Gmail SMTP credentials and an appropriate App Password.
- The current project is primarily a working prototype and may require additional scalability and production-level testing before large-scale deployment.

5. References / Literature Review Table

S. No	Source Title / Tool	Key Idea	Project Relevance
1	React.js Official Documentation	Component-based approach for developing reusable and interactive user interfaces	Used to develop the frontend interface of the AI Loan Management System.
2	Express.js Official Guide	Provides a lightweight framework for building server-side applications and REST APIs.	Used for developing the backend APIs and handling application requests.
3	PostgreSQL Official Documentation	Provides a reliable relational database system for structured and persistent data storage.	Used to store user, loan, financial, and authentication-related data.
4	Prisma ORM Documentation	Provides database modeling, migrations, and structured database access for Node.js applications.	Used as the ORM layer between the Node.js backend and PostgreSQL database.
5	JWT – JSON Web Tokens	Provides token-based authentication for securely identifying authenticated users.	Used for user authentication and protecting restricted application resources.
6	Google Gemini API Documentation	Enables applications to integrate generative AI capabilities for text generation and intelligent interactions.	Used to provide AI-powered financial assistance within the loan management system.
7	Nodemailer Documentation	Provides email-sending functionality for Node.js applications through SMTP and other transport mechanisms.	Used for application email services through Gmail SMTP.

5.1. References (APA Format)

Meta Platforms, Inc. (2024). *React documentation*. <https://react.dev/>

OpenJS Foundation. (2024). *Express.js documentation*. <https://expressjs.com/>

PostgreSQL Global Development Group. (2024). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>

Prisma. (2024). *Prisma ORM documentation*. <https://www.prisma.io/docs/orm/v6>

Auth0. (2024). *Introduction to JSON Web Tokens*. JWT.io. <https://jwt.io/introduction>

Google. (2024). *Gemini API documentation*. Google AI for Developers. <https://ai.google.dev/gemini-api/docs>

Nodemailer. (2024). *Nodemailer documentation*. <https://nodemailer.com/>